Documentação Técnica

Solução: EcoPulse (SoulUp)

Domain Driven Design using Java — 4º Sprint

Equipe: EcoPulse
Projeto: Challenge EcoPulse / SoulUp API RESTful
Data: 29/10/2026
Versão: 2.0 — Entrega Final (API Spring Boot + Deploy Cloud)

Integrantes

Pedro Henrique Carvalho do Nascimento — RM 570492
Gabriel Pereira de Oliveira — RM 572262
Adalto Massashiro Nagabe — RM 572298
Kauã Barros Ferreira — RM 571801
Murillo Siviero Lopes — RM 572724

Sumário

1 Objetivo e escopo do projeto 1
2 Principais funcionalidades da solução 1
2.1 Autenticação, segurança e gestão de usuários e famílias 1
2.2 Personalização e evolução de avatares (`Souly`) 1
2.3 Missões ambientais, catálogo de recompensas e participações 1
2.4 Registro de atividades sustentáveis e sequências (`Streaks`) 2
2.5 Comunidades, desafios coletivos e feed de publicações 2
2.6 Infraestrutura REST, `CorsFilter` e tratamento global de erros 2
3 Tabela de Endpoints (API RESTful) 3
4 Protótipo — Prints das telas implementadas 6
4.1 Tela inicial 6
4.2 Tela Sobre o EcoPulse 7
4.3 Tela Sobre o Souly 8
4.4 Tela Souly como guia interativo 9
4.5 Tela de integrantes 10
4.6 Tela de perguntas frequentes (FAQ) 11
4.7 Tela de contato 12
5 Arquitetura da solução e boas práticas 13
6 Modelo de Entidade-Relacionamento (MER) 13
6.1 Diagrama Entidade-Relacionamento (`T_SOUL_*`) 14
7 Diagrama de Classes atualizado 15
7.1 Diagrama de Classes das Camadas Resource, BO/Service, DAO e Configuração 16
8 Procedimentos para rodar a aplicação 17
8.1 Ferramentas e versões necessárias 17
8.2 Como importar o projeto na IDE 17
8.3 Configuração das variáveis de ambiente (Oracle FIAP) 17
8.4 Passo a passo para compilar e executar localmente 17
8.5 Acesso em Produção (Deploy Docker no Render) 18
8.6 Execução dos testes automatizados 18
9 Referências 18

1 Objetivo e escopo do projeto

O **EcoPulse (SoulUp)** é uma plataforma gamificada de incentivo a hábitos sustentáveis que conecta cidadãos, famílias e comunidades por meio de missões ambientais, acompanhamento de sequências diárias (**streaks**), recompensas e evolução do mascote virtual **Souly**.

Nesta entrega do **4º Sprint de Domain Driven Design Using Java**, a solução foi consolidada em uma **API RESTful completa** construída com **Java 21**, **Spring Boot 3.3.5** e **JDBC puro sobre Oracle Database** (padrão relacional consolidado `T_SOUL_*`), pronta para integração direta com o Front-End Web e publicada em nuvem via **Docker no Render** (`https://challengeecopulsemaven.onrender.com`).

O escopo implementado abrange:

- **Camada Modelo (`domain`):** 16 entidades de domínio que representam fielmente o modelo relacional `T_SOUL_*`, concentrando invariantes e comportamentos de negócio;
- **Camada DAO e BO (`repository/dao` e `service`):** persistência JDBC parametrizada (`PreparedStatement`) com CRUD completo, `ConnectionFactory` centralizada com suporte a transações ACID (`ThreadLocal`) e validações de regras de negócio;
- **Camada Resource (`resource`, `dto` e `config`):** 13 controladores REST sob `/api/v1`, contratos DTO segregados para proteção de dados sensíveis, filtro explícito `CorsFilter`, documentação interativa OpenAPI/Swagger e tratamento padronizado de exceções em `GlobalExceptionHandler`;
- **Deploy em Nuvem:** containerização **multi-stage** via `Dockerfile` e publicação contínua na plataforma Render.

2 Principais funcionalidades da solução

2.1 Autenticação, segurança e gestão de usuários e famílias

O sistema separa o nome público de exibição (`username`) da credencial única de acesso (`login`). As senhas nunca trafegam nem são armazenadas em texto puro: o `PasswordHasher` aplica **BCrypt** antes da gravação em `T_SOUL_USUARIO`. Os contratos de saída separam `UserPublicResponse` (listagens públicas sem CPF ou data de nascimento) de `UserDetailResponse` (restrito ao próprio titular mediante cabeçalho `X-User-Id`), ocultando sempre `passwordHash`. O módulo de famílias (`T_SOUL_FAMILIA`) agrupa usuários e permite consultar todos os membros vinculados.

2.2 Personalização e evolução de avatares (`Souly`)

Permite cadastrar avatares personalizados (`visualConfig` e `personalityType`) vinculados a usuários previamente existentes em `T_SOUL_USUARIO`. A entidade `Avatar` gerencia a progressão gamificada em `T_SOUL_AVATAR`: ao receber pontos de experiência (`PATCH /api/v1/avatars/{id}/experience`), o nível (`level`) sobe automaticamente a cada 1.000 XP acumulados e sinaliza o estado de evolução (`isEvolved`).

2.3 Missões ambientais, catálogo de recompensas e participações

As antigas estruturas de missão e desafio foram unificadas na entidade `Mission` (`T_SOUL_DESAFIO`), com controle único de status (`ATIVA` ou `INATIVA`), regras (`rules`), data de início obrigatória e data de término opcional (`isEligible`). O catálogo de recompensas (`Reward` — `T_SOUL_REWARD`) é independente e relaciona-se em cardinalidade N:N via `T_SOUL_DESAFIO_REWARD`:

- **Invariante transacional de criação:** toda missão exige ao menos uma recompensa previamente cadastrada; caso qualquer ID informado não exista, toda a transação é revertida (**rollback**);
- **Proteção de integridade:** o `MissionService` bloqueia a remoção da última recompensa associada a uma missão;

- **Conclusão e premiação automática:** em `UserMissionService`, o usuário se inscreve em missões elegíveis (`0%` a `100%` de progresso) e, ao concluir a participação (`POST /api/v1/participations/{id}/complete`), recebe automaticamente na mesma transação todas as recompensas vinculadas em `T_SOUL_USUARIO_REWARD`, podendo resgatá-las via `PATCH /api/v1/user-rewards/{id}/claim`.

2.4 Registro de atividades sustentáveis e sequências (`Streaks`)

Cada ação ecológica registrada em `ActivityService` (`POST /api/v1/activities`, como reciclagem, compostagem ou economia de água) persiste o evento em `T_SOUL_ACTIVITY` e atualiza transacionalmente o `Streak` correspondente do usuário em `T_SOUL_STREAK` (considerando o fuso `America/Sao_Paulo`): incrementa a sequência em dias consecutivos, mantém no mesmo dia ou reinicia para `1` após quebra de sequência, preservando sempre o recorde histórico (`highestCount`).

2.5 Comunidades, desafios coletivos e feed de publicações

O módulo de comunidades (`T_SOUL_COMUNIDADE`) gerencia grupos temáticos com nome único, associação de membros (`T_SOUL_USUARIO_COMUN`) e vínculo de missões coletivas com placar de pontuação (`MissionCommunity` — `T_SOUL_DESAFIO_COMUN`). Já o feed social (`Post` — `T_SOUL_POSTS`) inicializa atomicamente seu registro 1:1 de métricas (`PostCounter` — `T_SOUL_CONTADORES`) com contadores zerados, incrementando curtidas, comentários e compartilhamentos exclusivamente por eventos de servidor. A entidade `ConversationHistory` (`T_SOUL_HISTORICO_CONVERSA`) mantém a estrutura relacional pronta no banco sem expor endpoints de chat/IA nesta fase.

2.6 Infraestrutura REST, `CorsFilter` e tratamento global de erros

A classe `CorsFilter` implementa `jakarta.servlet.Filter` com precedência máxima, habilitando requisições **cross-origin** e respostas imediatas `200 OK` a **preflights** `OPTIONS` do Front-End. O `GlobalExceptionHandler` (`@RestControllerAdvice`) intercepta exceções de domínio, Bean Validation, autenticação, conflito e banco de dados, devolvendo respostas JSON padronizadas (`ApiErrorResponse`) com os códigos HTTP semânticos adequados.

3 Tabela de Endpoints (API RESTful)

Todos os recursos seguem os princípios REST sob o prefixo `/api/v1`, operando com payloads JSON (`application/json`) em padrão ISO-8601. A documentação interativa **Swagger UI** fica disponível em `/swagger-ui.html`.

Recurso	Verbo	URI	Descrição	Status HTTP
Health	GET	/api/v1/health	Verifica disponibilidade da API no Render.	200
Auth	POST	/api/v1/auth/register	Cadastra usuário com senha hash BCrypt.	201, 400, 404, 409
Auth	POST	/api/v1/auth/login	Autentica por login e password.	200, 400, 401
Auth	GET	/api/v1/auth/me	Retorna perfil autenticado (X-User-Id).	200, 401, 404
Usuários	POST	/api/v1/users	Cria novo usuário validando CPF, login e e-mail.	201, 400, 404, 409
Usuários	GET	/api/v1/users	Lista usuários públicos (filtro opcional ?username=).	200
Usuários	GET	/api/v1/users/{id}	Consulta perfil público (sem CPF/nascimento).	200, 400, 404
Usuários	GET	/api/v1/users/{id}/profile	Consulta perfil completo restrito ao titular.	200, 401, 404
Usuários	PUT	/api/v1/users/{id}	Atualiza dados cadastrais e família do usuário.	200, 400, 404, 409
Usuários	DELETE	/api/v1/users/{id}	Remove usuário por identificador.	204, 404, 500
Famílias	POST	/api/v1/families	Cadastra nova família (T_SOUL_FAMILIA).	201, 400
Famílias	GET	/api/v1/families	Lista todas as famílias cadastradas.	200
Famílias	GET	/api/v1/families/{id}	Busca família por ID.	200, 404
Famílias	GET	/api/v1/families/{id}/members	Lista usuários vinculados à família.	200, 404
Famílias	PUT	/api/v1/families/{id}	Atualiza nome e descrição de parentesco.	200, 400, 404
Famílias	DELETE	/api/v1/families/{id}	Exclui família (se não possuir vínculos ativos).	204, 404, 500
Avatares	POST	/api/v1/avatars	Cria avatar vinculado a usuário existente.	201, 400, 404
Avatares	GET	/api/v1/avatars	Lista todos os avatares.	200
Avatares	GET	/api/v1/avatars/user/{userId}	Lista avatares pertencentes a um usuário.	200
Avatares	GET	/api/v1/avatars/{id}	Consulta detalhes e XP do avatar.	200, 404
Avatares	PUT	/api/v1/avatars/{id}	Atualiza nome, visualConfig e personalidade.	200, 400, 404
Avatares	PATCH	/api/v1/avatars/{id}/experience	Adiciona XP e recalcula evolução de nível.	200, 400, 404
Avatares	DELETE	/api/v1/avatars/{id}	Remove avatar por identificador.	204, 404
Recompensas	POST	/api/v1/rewards	Cria recompensa no catálogo (T_SOUL_REWARD).	201, 400
Recompensas	GET	/api/v1/rewards	Lista catálogo de recompensas.	200
Recompensas	GET	/api/v1/rewards/{id}	Busca recompensa por ID.	200, 404

Recurso	Verbo	URI	Descrição	Status HTTP
Recompensas	PUT	/api/v1/rewards/{id}	Atualiza dados e pontuação da recompensa.	200, 400, 404
Recompensas	DELETE	/api/v1/rewards/{id}	Exclui recompensa do catálogo.	204, 404, 500
Missões	POST	/api/v1/missions	Cria missão exigindo >= 1 recompensa prévia.	201, 400, 404
Missões	GET	/api/v1/missions	Lista missões (?activeOnly= / ?eligibleOnly=).	200
Missões	GET	/api/v1/missions/{id}	Consulta missão e suas recompensas vinculadas.	200, 404
Missões	PUT	/api/v1/missions/{id}	Atualiza título, tipo, regras e período.	200, 400, 404
Missões	PATCH	/api/v1/missions/{id}/status	Altera status entre ATIVA e INATIVA.	200, 400, 404
Missões	GET	/api/v1/missions/{id}/rewards	Lista recompensas associadas à missão.	200, 404
Missões	POST	/api/v1/missions/{id}/rewards/{rewardId}	Associa nova recompensa à missão.	201, 404, 409
Missões	DELETE	/api/v1/missions/{id}/rewards/{rewardId}	Remove vínculo (bloqueia se for a última).	204, 400, 404
Missões	DELETE	/api/v1/missions/{id}	Remove missão e associações em transação.	204, 404, 500
Participações	POST	/api/v1/participations	Inscreve usuário em missão elegível.	201, 400, 404, 409
Participações	GET	/api/v1/participations	Lista participações (?userId= opcional).	200
Participações	GET	/api/v1/participations/{id}	Consulta progresso da participação.	200, 404
Participações	PATCH	/api/v1/participations/{id}/progress	Atualiza percentual (0 a 100%).	200, 400, 404
Participações	POST	/api/v1/participations/{id}/complete	Conclui missão e concede recompensas.	200, 404, 409
Participações	DELETE	/api/v1/participations/{id}	Cancela/exclui registro de participação.	204, 404
User Rewards	POST	/api/v1/user-rewards	Concede recompensa avulsa ao usuário.	201, 400, 404
User Rewards	GET	/api/v1/user-rewards	Lista conquistas (?userId= opcional).	200
User Rewards	GET	/api/v1/user-rewards/{id}	Consulta conquista específica.	200, 404
User Rewards	PATCH	/api/v1/user-rewards/{id}/claim	Marca recompensa como resgatada (true).	200, 404, 409
User Rewards	DELETE	/api/v1/user-rewards/{id}	Remove registro de recompensa adquirida.	204, 404
Atividades	POST	/api/v1/activities	Registra hábito e atualiza Streak diário.	201, 400, 404
Atividades	GET	/api/v1/activities	Lista atividades (?userId= opcional).	200
Atividades	GET	/api/v1/activities/{id}	Consulta atividade ecológica por ID.	200, 404
Atividades	DELETE	/api/v1/activities/{id}	Remove atividade registrada.	204, 404
Streaks	GET	/api/v1/streaks	Lista todas as sequências ativas.	200
Streaks	GET	/api/v1/streaks/user/{userId}	Lista sequências e recordes de um usuário.	200

Recurso	Verbo	URI	Descrição	Status HTTP
Streaks	GET	/api/v1/streaks/{id}	Consulta sequência específica por ID.	200, 404
Comunidades	POST	/api/v1/communities	Cria comunidade com nome único.	201, 400, 409
Comunidades	GET	/api/v1/communities	Lista comunidades cadastradas.	200
Comunidades	GET	/api/v1/communities/{id}	Consulta comunidade por ID.	200, 404
Comunidades	PUT	/api/v1/communities/{id}	Atualiza dados da comunidade.	200, 400, 404, 409
Comunidades	DELETE	/api/v1/communities/{id}	Remove comunidade e vínculos.	204, 404
Comunidades	GET	/api/v1/communities/{id}/members	Lista usuários membros da comunidade.	200, 404
Comunidades	POST	/api/v1/communities/{id}/members/{userId}	Adiciona membro à comunidade.	201, 404, 409
Comunidades	DELETE	/api/v1/communities/{id}/members/{userId}	Remove membro da comunidade.	204, 404
Comunidades	GET	/api/v1/communities/{id}/missions	Lista desafios vinculados à comunidade.	200, 404
Comunidades	POST	/api/v1/communities/{id}/missions	Vincula missão à comunidade com placar.	201, 400, 404, 409
Comunidades	PUT	/api/v1/communities/{id}/missions/{missionId}	Atualiza pontuação/status na comunidade.	200, 400, 404
Comunidades	DELETE	/api/v1/communities/{id}/missions/{missionId}	Desvincula missão da comunidade.	204, 404
Publicações	POST	/api/v1/posts	Cria post e inicializa PostCounter (1:1).	201, 400, 404
Publicações	GET	/api/v1/posts	Lista feed de posts (?userId= opcional).	200
Publicações	GET	/api/v1/posts/{id}	Consulta publicação com contadores.	200, 404
Publicações	PUT	/api/v1/posts/{id}	Edita conteúdo e status da publicação.	200, 400, 404
Publicações	DELETE	/api/v1/posts/{id}	Exclui publicação e seus contadores.	204, 404
Publicações	GET	/api/v1/posts/{id}/counters	Consulta métricas 1:1 do post.	200, 404
Publicações	POST	/api/v1/posts/{id}/like	Incrementa curtidas via evento no servidor.	200, 404
Publicações	POST	/api/v1/posts/{id}/comment	Incrementa contagem de comentários.	200, 404
Publicações	POST	/api/v1/posts/{id}/share	Incrementa contagem de compartilhamentos.	200, 404

4 Protótipo — Prints das telas implementadas

As capturas de tela reais abaixo apresentam o Front-End Web desenvolvido em React e Vite para o ecossistema **EcoPulse / SoulUp**, bem como o fluxo visual suportado pelos endpoints REST (`/api/v1`) e pelo filtro `CorsFilter` implementados no Back-End Java.

4.1 Tela inicial

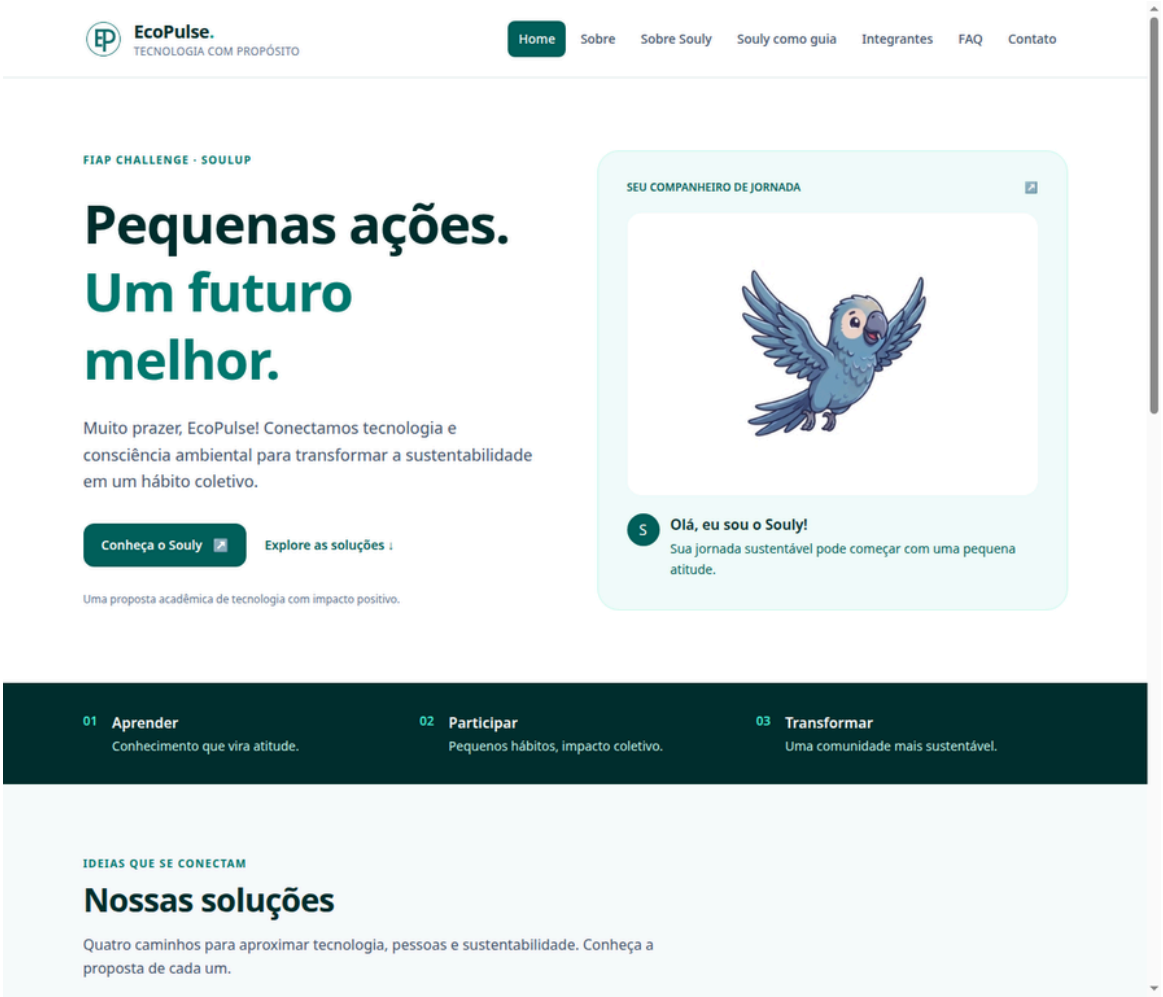


Figura 1: Tela inicial da plataforma EcoPulse — rota `/`.

Apresenta a proposta de valor do EcoPulse, o mascote interativo **Souly** e os três pilares da jornada gamificada (**Aprender**, **Participar** e **Transformar**), conectados no Back-End aos módulos de missões (`/api/v1/missions`), atividades (`/api/v1/activities`) e comunidades (`/api/v1/communities`).

4.2 Tela Sobre o EcoPulse



Figura 2: Apresentação institucional de propósito, missão, visão e valores — rota `/sobre`.

Detalha o problema socioambiental abordado no **SoulUp Challenge** e como pequenas atitudes cotidianas geram impacto coletivo mensurável e recompensas reais dentro da plataforma.

4.3 Tela Sobre o Souly

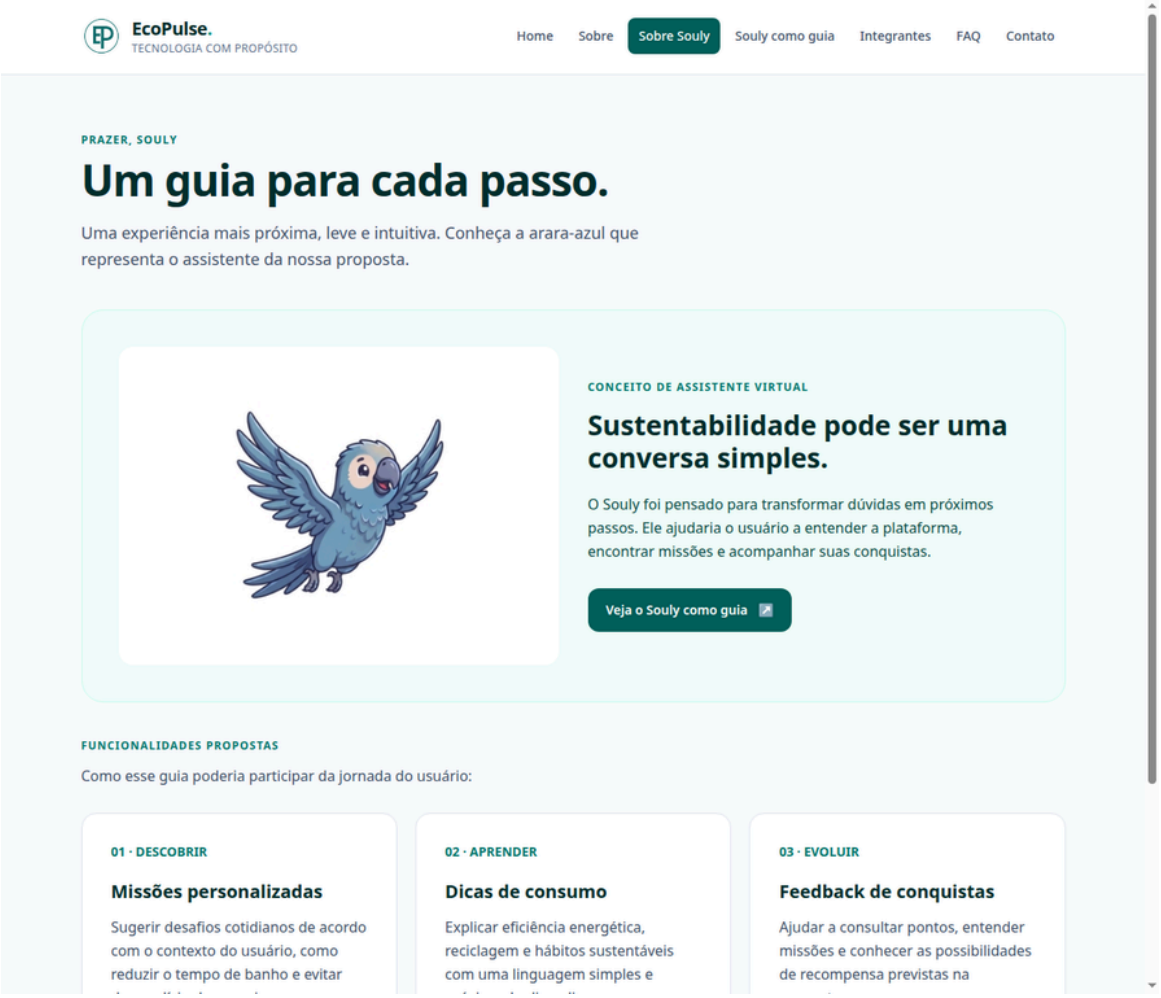


Figura 3: Apresentação do avatar e guia Souly — rota `/sobre-souly`.

Explica a dinâmica de acompanhamento do avatar **Souly** (arara-azul), cuja evolução de nível, experiência (XP), personalidade e configuração visual são gerenciadas pelos endpoints de `/api/v1/avatars`.

4.4 Tela Souly como guia interativo

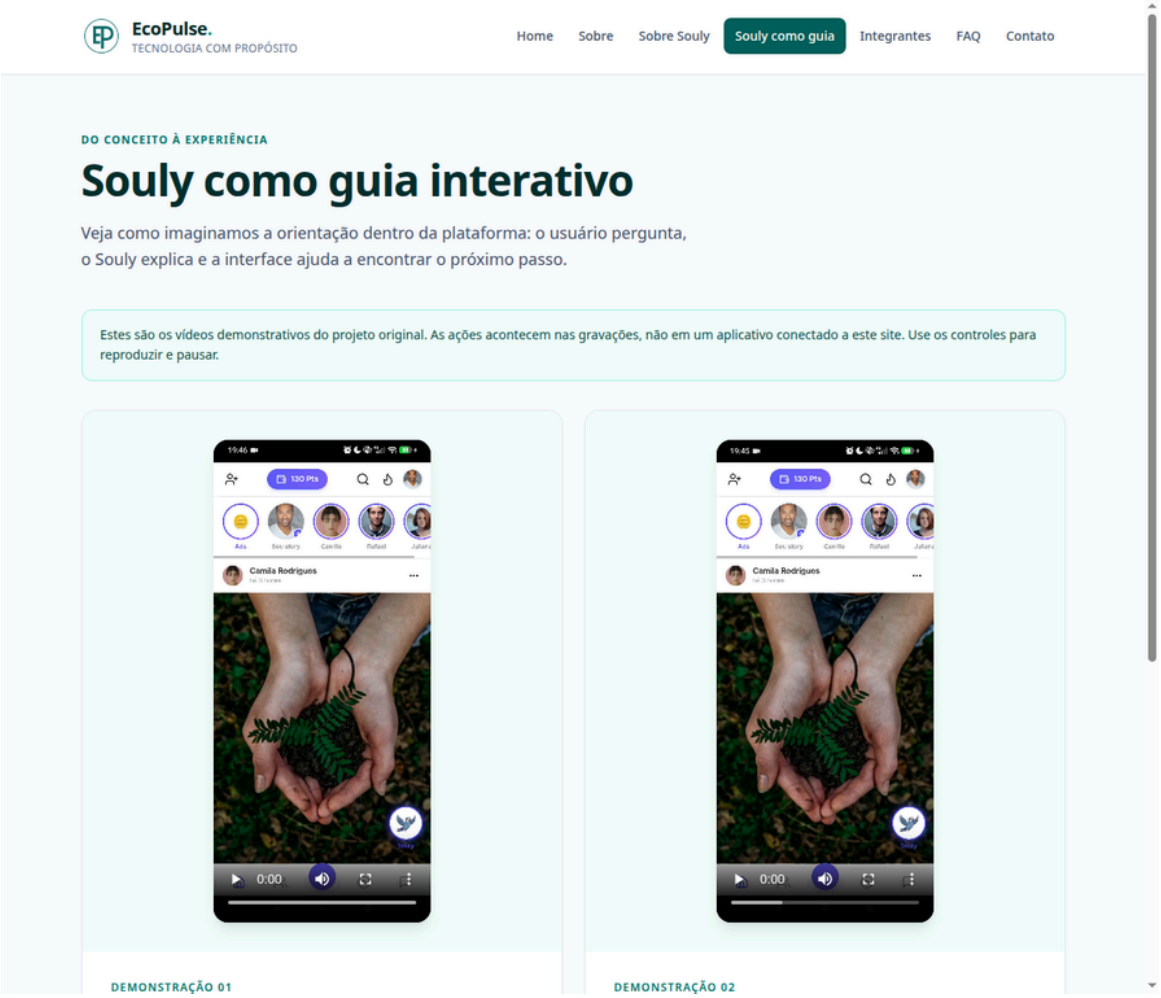


Figura 4: Demonstrações da jornada mobile/web de missões e conquistas — rota `/souly-como-guia`. Demonstra a experiência do usuário ao consultar desafios ambientais, acompanhar pontuação e visualizar recompensas conquistadas — operações sustentadas na API por `/api/v1/participations` e `/api/v1/user-rewards`.

4.5 Tela de integrantes

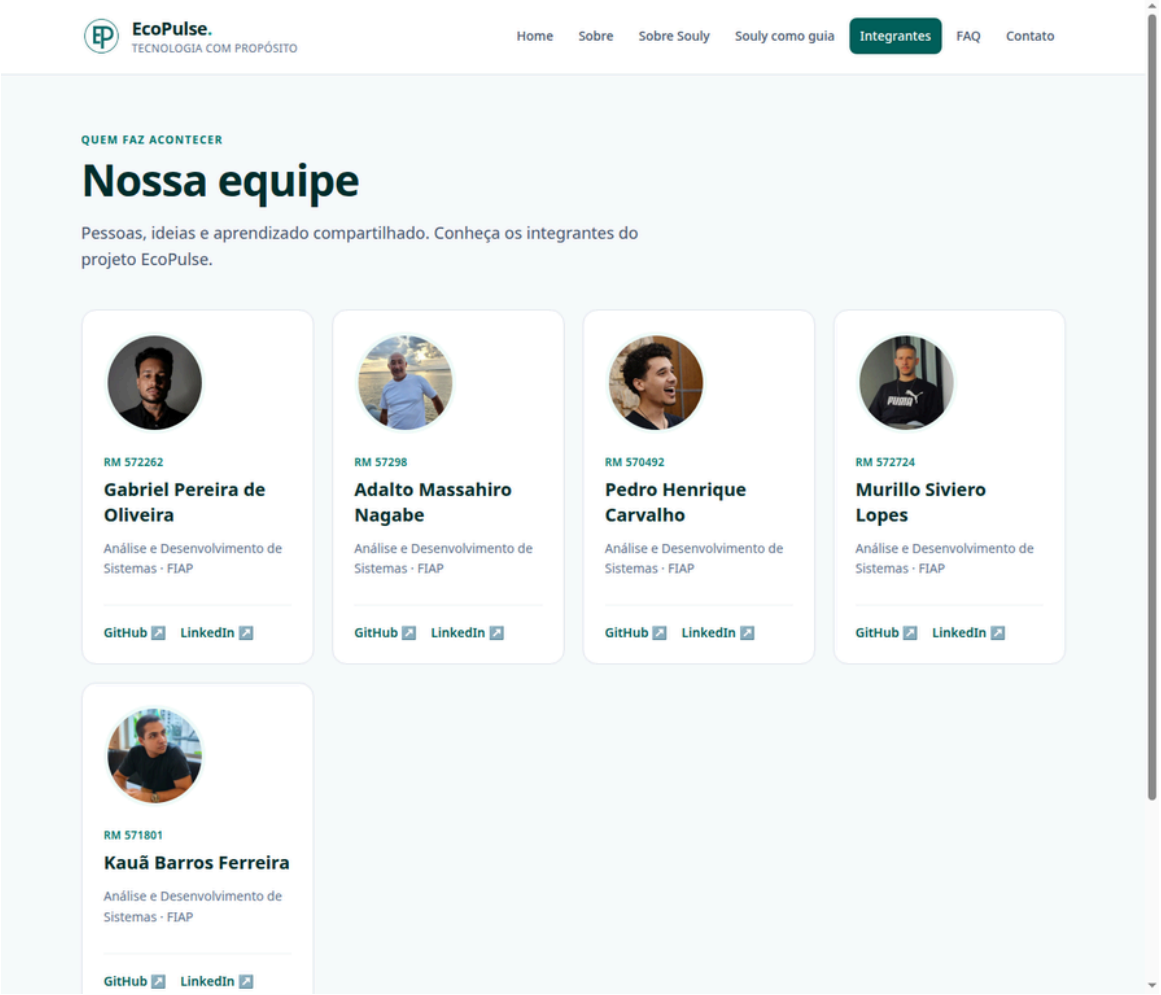


Figura 5: Tela de integrantes da equipe EcoPulse — rota `/integrantes`.

Exibe os cinco integrantes responsáveis pela arquitetura e desenvolvimento do projeto na FIAP, com seus respectivos RMs e links diretos para GitHub e LinkedIn.

4.6 Tela de perguntas frequentes (FAQ)

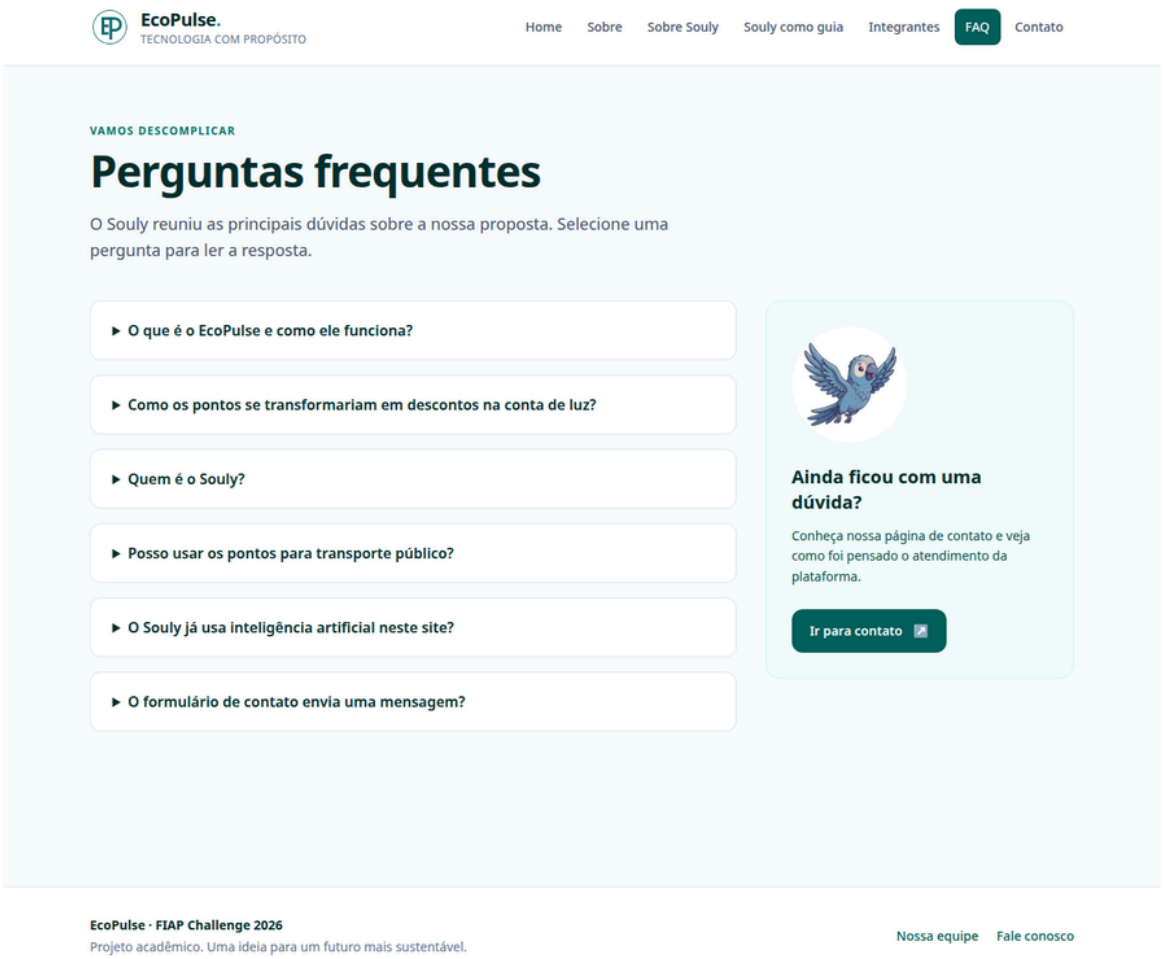


Figura 6: Componente expansível de dúvidas frequentes — rota `/faq`.

Organiza em acordeões interativos as principais dúvidas sobre conversão de pontos, funcionamento das missões sustentáveis, comunidades e recompensas.

4.7 Tela de contato

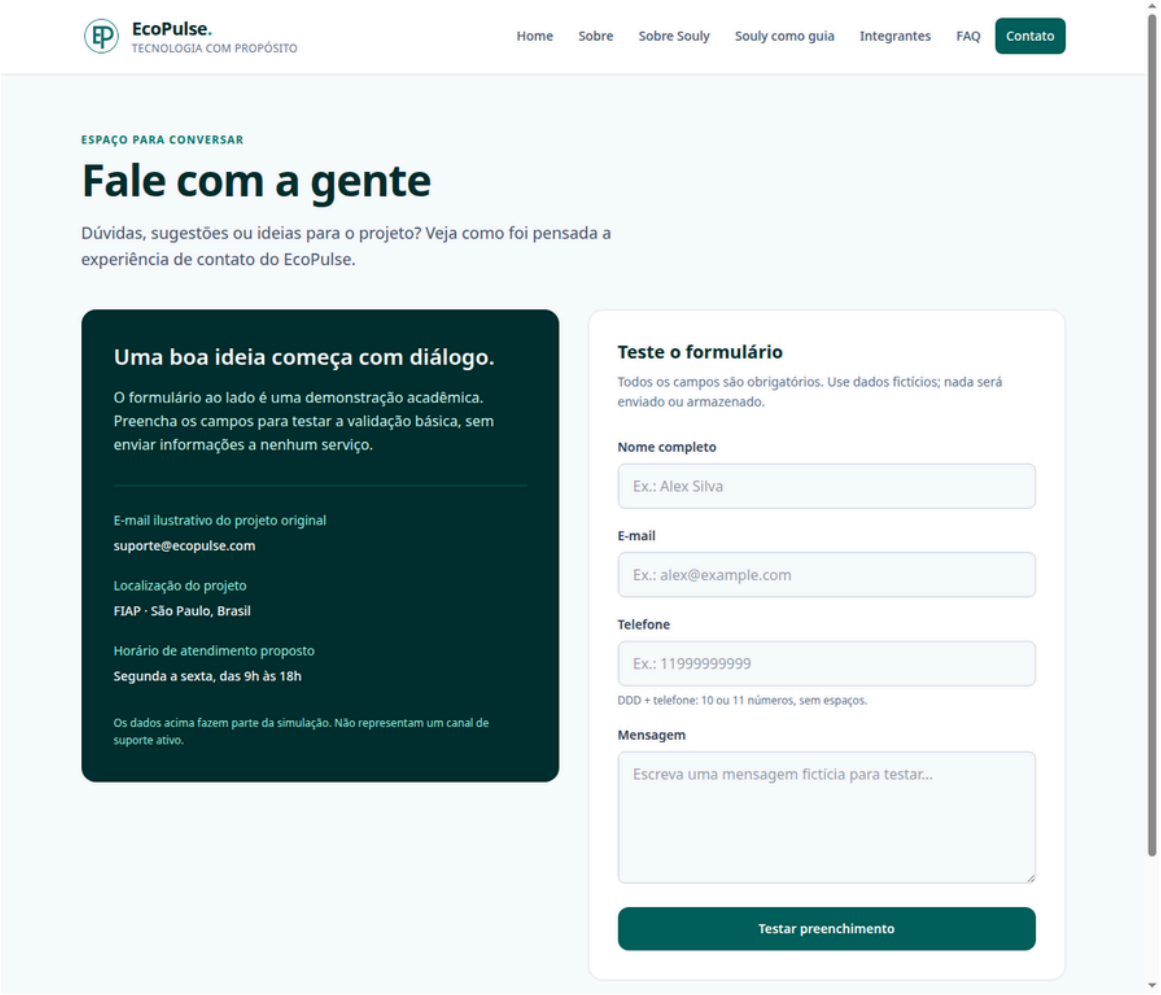


Figura 7: Formulário validado de atendimento e contato — rota `/contato`.

Apresenta os canais institucionais da equipe e formulário com validação de campos no cliente para envio de dúvidas e sugestões.

5 Arquitetura da solução e boas práticas

O projeto segue os princípios de **Domain Driven Design (DDD)** em camadas bem definidas sob o pacote `br.com.EcoPulse`, aplicando os padrões **DAO**, **BO/Service**, **Connection Factory**, **DTO**, **Resource REST** e **Filter**:

Camada / Pacote	Responsabilidade no Projeto
`domain` (Modelo)	16 entidades de domínio (`User`, `Family`, `Avatar`, `Mission`, `Reward`, `UserMission`, `UserReward`, `Activity`, `Streak`, `Community`, `UserCommunity`, `MissionCommunity`, `Post`, `PostCounter`, `ConversationHistory`) com validações e regras encapsuladas.
`repository` e `dao`	Interface genérica `GenericRepository`, repositórios e DAOs JDBC concretos com SQL parametrizado (`PreparedStatement`) mapeando todas as tabelas `T_SOUL_*`.
`service` (BO)	Orquestração de regras de negócio, unicidade cadastral, cálculo de `Streak`, concessão automática de recompensas e transações atômicas via `ConnectionFactory.executeInTransaction`.
`resource` e `dto`	13 controllers REST (`/api/v1/*`) seguindo verbos e códigos HTTP semânticos, desacoplados do modelo interno por Records DTO com Bean Validation (`@Valid`).
`config` e `security`	`ConnectionFactory` externalizada por variáveis de ambiente, `CorsFilter` explícito para integração Front-End, `OpenApiConfig` (Swagger) e `PasswordHasher` (BCrypt).
`exception`	Hierarquia `EcoPulseException` (`DomainValidationException`, `ResourceNotFoundException`, `BusinessConflictException`, `AuthenticationException`, `PersistenceException`) tratada globalmente em `GlobalExceptionHandler`.
`controller` / `interfaces`	Camada de compatibilidade preservada para execução opcional via console interativo (`Exibition`).

6 Modelo de Entidade-Relacionamento (MER)

O banco de dados relacional no Oracle utiliza o padrão consolidado de nomenclatura `T_SOUL_*` composto por **16 tabelas** com integridade referencial completa (`PK`, `FK`, `UK` e `CHECK`). A tabela central `T_SOUL_USUARIO` conecta-se opcionalmente a `T_SOUL_FAMILIA` (`N:1`) e origina os vínculos com avatares (`T_SOUL_AVATAR`), atividades (`T_SOUL_ACTIVITY`), sequências (`T_SOUL_STREAK`), participações em desafios (`T_SOUL_USUARIO_DESAFIO`), recompensas adquiridas (`T_SOUL_USUARIO_REWARD`), comunidades (`T_SOUL_USUARIO_COMUN`), publicações (`T_SOUL_POSTS`, ligada `1:1` a `T_SOUL_CONTADORES`) e histórico estrutural (`T_SOUL_HISTORICO_CONVERSA`). Os desafios (`T_SOUL_DESAFIO`) associam-se às recompensas (`T_SOUL_REWARD`) por `T_SOUL_DESAFIO_REWARD` e às comunidades por `T_SOUL_DESAFIO_COMUN`.

6.1 Diagrama Entidade-Relacionamento (`T_SOUL_*`)

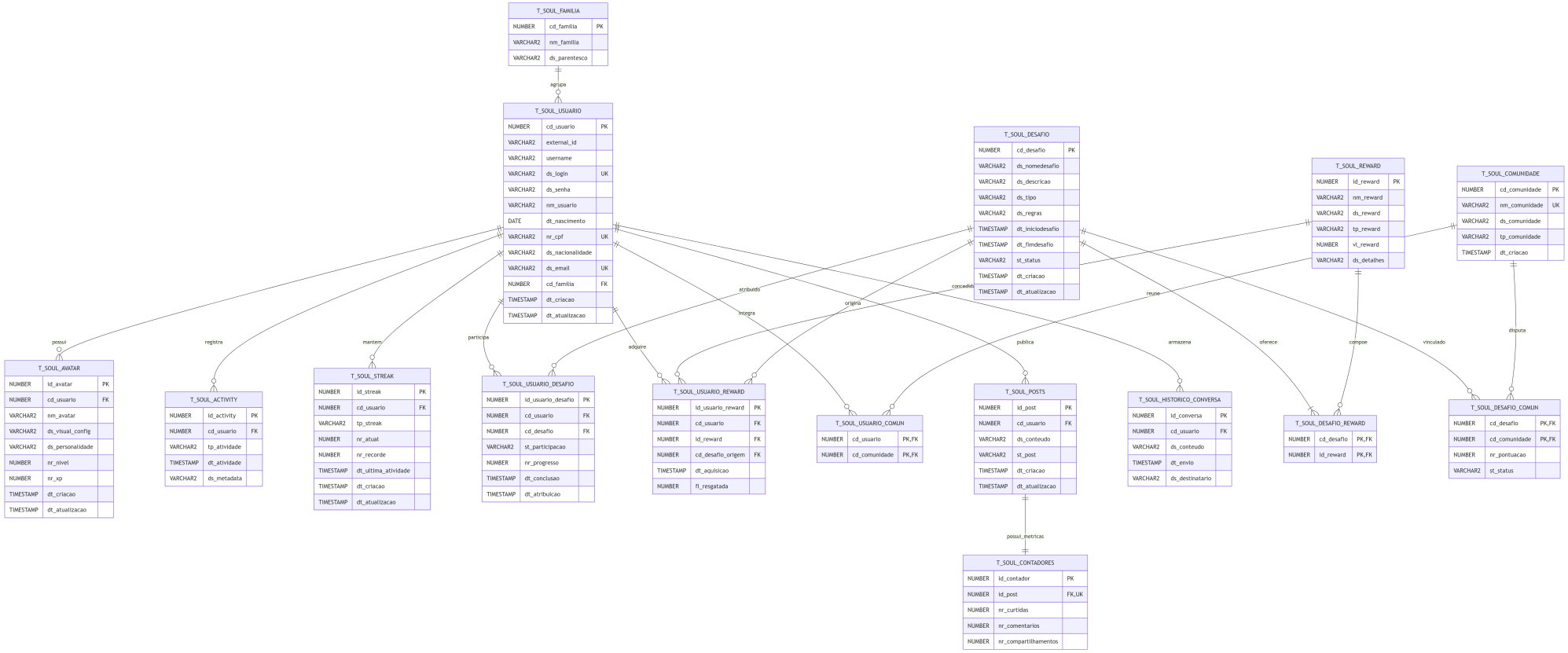


Figura 8: Modelo Entidade-Relacionamento (MER) consolidado das 16 tabelas `T_SOUL_*` no Oracle Database.

7 Diagrama de Classes atualizado

Os diagramas a seguir refletem fielmente o código-fonte Java entregue na 4ª Sprint. O primeiro detalha todas as classes da camada `domain` (atributos, métodos de negócio, composições e multiplicidades) e a hierarquia de herança das exceções (`EcoPulseException`). O segundo apresenta a integração entre as camadas `resource`, `service` (BO), `repository/dao`, `CorsFilter` e `ConnectionFactory`.

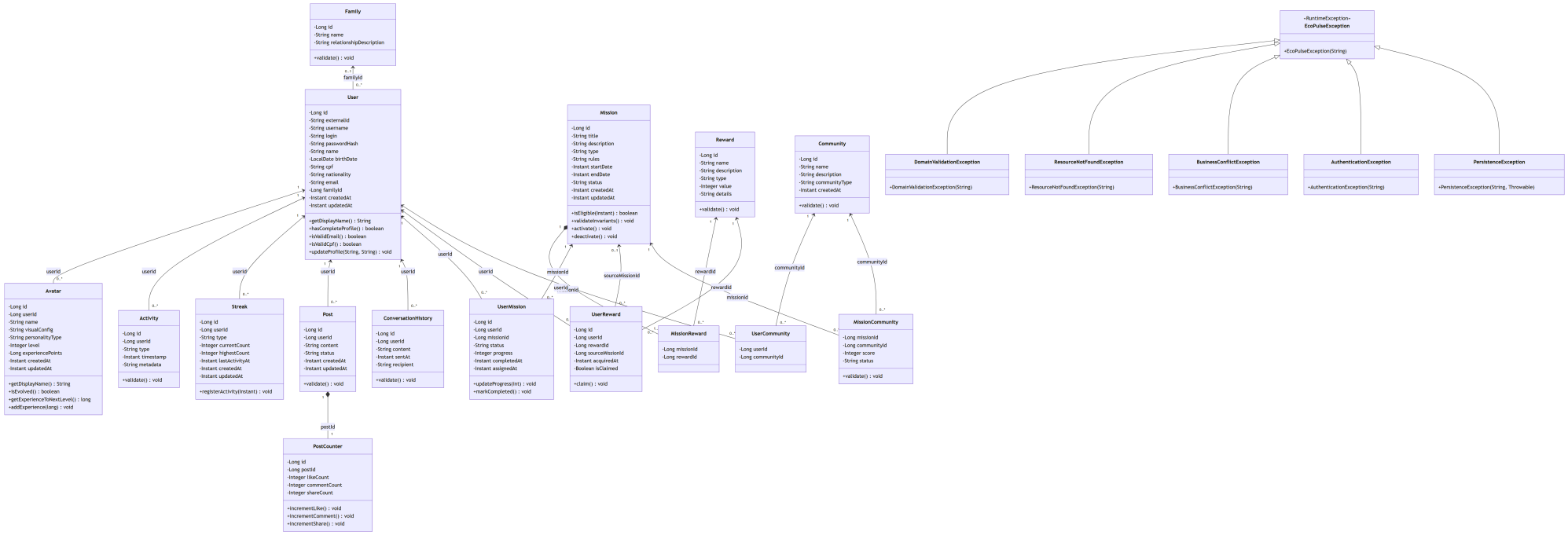


Figura 9: Diagrama de classes da camada de Domínio (`br.com.EcoPulse.domain`) e herança de Exceções (`br.com.EcoPulse.exception`).

7.1 Diagrama de Classes das Camadas Resource, BO/Service, DAO e Configuração

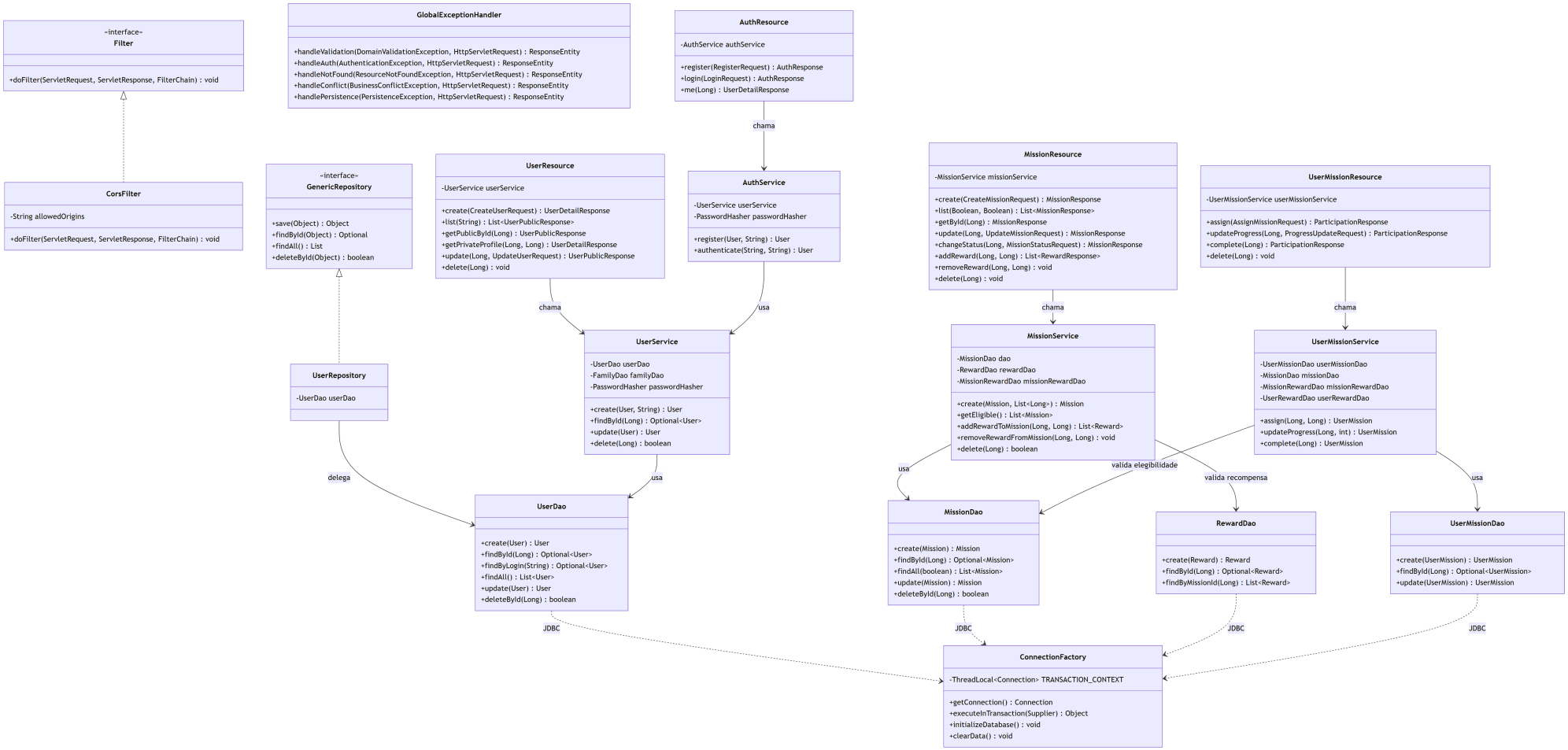


Figura 10: Diagrama de classes representando a comunicação entre Resources REST, Services (BO), DAOs/Repositories, `CorsFilter` e `ConnectionFactory`.

8 Procedimentos para rodar a aplicação

8.1 Ferramentas e versões necessárias

Ferramenta / Componente	Versão e Finalidade
Java Development Kit (JDK)	**JDK 21** (LTS) — compilação e execução da aplicação.
Apache Maven	**Maven 3.9+** — gerenciamento de dependências, testes e empacotamento JAR.
Spring Boot	**3.3.5** (`spring-boot-starter-web`, `validation`, `spring-security-crypto`).
IDE Recomendada	**IntelliJ IDEA** (compatível também com **Eclipse**, **NetBeans** e **VS Code** com extensão Java/Maven).
Banco de Dados Principal	**Oracle Database FIAP** (`jdbc:oracle:thin:@oracle.fiap.com.br:1521:ORCL`) via driver `ojdbc11:23.5.0.24.07`.
Banco Auxiliar Local/Testes	**H2 Database** (em memória com `MODE=Oracle`) para execução rápida com perfil `dev` ou `test`.
Container e Nuvem	**Docker** (build multi-stage) e plataforma **Render** para deploy em produção.

8.2 Como importar o projeto na IDE

1. Extraia ou clone o repositório `ChallengeEcopulseMaven` em uma pasta local.
2. No **IntelliJ IDEA**, clique em **File -> Open** e selecione a pasta raiz do projeto (onde está localizado o arquivo `pom.xml`). **Em Eclipse ou NetBeans**, utilize **Import -> Existing Maven Projects** apontando para a mesma raiz; no **VS Code**, abra a pasta raiz com o **Extension Pack for Java** instalado.
3. No painel lateral do Maven, clique em **Reload All Maven Projects** para baixar as dependências e configurar o SDK do projeto como **Java 21**.

8.3 Configuração das variáveis de ambiente (Oracle FIAP)

Por boas práticas de segurança, as credenciais do banco não ficam fixas no código-fonte. Crie um arquivo `.env` na raiz (ou configure em **Run/Debug Configurations -> Environment variables** na IDE) com os parâmetros abaixo:

```
PORT=8080
DB_URL=jdbc:oracle:thin:@oracle.fiap.com.br:1521:ORCL
DB_USER=rm570492
DB_PASSWORD=<SUA_SENHA_ORACLE_FIAP>
DB_DRIVER=oracle.jdbc.OracleDriver
DB_AUTO_INIT=true
CORS_ALLOWED_ORIGINS=*
```

Na inicialização, a `ConnectionFactory` verifica o schema e cria automaticamente todas as 16 tabelas `T_SOUL_*` caso ainda não existam.

8.4 Passo a passo para compilar e executar localmente

Opção 1 — Executar conectado ao Oracle da FIAP:

1. Com as variáveis `DB_URL`, `DB_USER` e `DB_PASSWORD` definidas no terminal ou na IDE, gere o pacote executável na raiz do projeto:

```
mvn clean package -DskipTests
```

2. Inicie a aplicação Spring Boot pela classe `br.com.EcoPulse.EcoPulseApplication` na IDE ou via terminal:

```
java -jar target/ecopulse-api.jar
```

Opção 2 — Executar imediatamente sem rede Oracle (perfil `dev` com H2 em modo Oracle):

```
java -jar target/ecopulse-api.jar --spring.profiles.active=dev
```

Validação local após iniciar o servidor (`http://localhost:8080`):

- **Healthcheck:** `http://localhost:8080/api/v1/health`
- **Documentação e testes interativos (Swagger UI):** `http://localhost:8080/swagger-ui.html`
- **OpenAPI JSON:** `http://localhost:8080/api-docs`

8.5 Acesso em Produção (Deploy Docker no Render)

A aplicação encontra-se implantada em container Docker (`Dockerfile` multi-stage com `maven:3.9.9-eclipse-temurin-21` e runtime `eclipse-temurin:21-jre-alpine`) na plataforma **Render**:

- **URL Base da API em Produção:** https://challengeecopulsemaven.onrender.com
- **Healthcheck Público:** https://challengeecopulsemaven.onrender.com/api/v1/health
- **Swagger UI em Produção:** https://challengeecopulsemaven.onrender.com/swagger-ui.html

8.6 Execução dos testes automatizados

Para executar os testes unitários de domínio (`UserTest`, `AvatarTest`, `MissionTest`) e o teste integrado REST (`EcoPulseApiIntegrationTest` com `MockMvc` sobre H2 em modo Oracle):

```
mvn clean test
```

9 Referências

[1] Spring Framework / Spring Boot 3.3 Reference Documentation: https://docs.spring.io/spring-boot/index.html

[2] Oracle Database JDBC Developer's Guide (`ojdbc11`): https://docs.oracle.com/en/database/oracle/oracle-database/23/jjdbc/

[3] Springdoc OpenAPI v2 (Swagger UI para Spring Boot 3): https://springdoc.org/

[4] Render Documentation — Deploying Docker Web Services: https://render.com/docs/docker

[5] JUnit 5 User Guide: https://junit.org/junit5/docs/current/user-guide/